Abstract

This project presents an autonomous chess-playing robot system that integrates computer vision, chess logic, a chess engine, and robotic motion planning. The system starts with camera calibration using four ArUco markers placed on the chessboard. After calibration, a YOLO-based vision model detects each chess piece and estimates its position relative to the chessboard frame. These detected positions are then mapped to internal chessboard coordinates, such as identifying a white pawn on e2.

A mapper and state-change node compares the current mapped board with the previous board state to detect the human move, such as e2e4. The detected move is sent to the chess interface, which updates the graphical board and publishes the FEN representation of the current position. The game engine receives the FEN and calculates the best move. This best move is sent back to the interface for visualization and to the robot commander for execution.

The robot commander receives the chess move, converts the source and target squares into Cartesian poses according to the robot chessboard frame, and executes a pick-and-place sequence using MoveIt. The robot system also includes a startup calibration step, where the robot moves to A1 for manual board alignment and waits for confirmation on the calibration topic before starting normal operation. Capture handling is performed by checking the board FEN before executing the move. If the destination square is occupied, the robot removes the captured piece first, drops it outside the board, returns home, and then executes the normal move.

Keywords: ROS2, YOLO, Stockfish, python-chess, ArUco, UR5e, MoveIt, Robotiq gripper, chess robot, computer vision.

Contents

1 Introduction **1**

1.1 Project Motivation 1

1.2 Project Objective 1

1.3 System Scope 2

2 System Overview **3**

2.1 System Workflow 3

2.2 High-Level Pipeline 3

2.3 System Demonstration 3

3 Hardware Setup **5**

3.1 Full Hardware Setup 5

3.2 Camera Setup 6

3.3 Chessboard Setup 7

3.4 Chessboard with ArUco Markers 7

3.5 Before and After Move 8

3.6 Chess Pieces 8

4 Coordinate Frames and Calibration **9**

4.1 Coordinate Frames 9

4.2 Chessboard Frame 9

4.3 Camera Calibration Using Four ArUco Markers 9

4.4 Manual Robot-to-Board Calibration 10

4.5 Startup Calibration Confirmation 10

5 Vision System and Mapping Architecture **11**

5.1 Vision Pipeline 11

5.2 YOLO Detection Node 11

5.3 YOLO Dataset and Training Setup 12

5.4 YOLO Detection Result 13

5.5 Coordinate Mapping and Boundary Clamping 14

6 Mapper and State-Change Detection **15**
6.1 Mapper Concept . . . 15
6.2 Mapper Example . . . 15
6.3 State-Change Detection . . . 16
6.4 Mapper and State-Change Terminal Result . . . 16
6.5 Robot Status Synchronization . . . 17

7 Chess Interface and Game Engine **18**
7.1 Chess Interface . . . 18
7.2 Chess GUI Result . . . 19
7.3 FEN Representation . . . 19
7.4 Stockfish Game Engine . . . 20

8 Robot System **21**
8.1 Robot Commander Node . . . 21
8.1.1 Main Node . . . 21
8.1.2 Subscribed Topics . . . 21
8.1.3 Published Topic . . . 21
8.2 Robot Setup in RViz . . . 21
8.3 Robot Startup Calibration . . . 22
8.4 Square-to-Pose Mapping . . . 23
8.5 Normal Robot Move Execution . . . 23
8.6 Capture Handling . . . 24
8.7 Captured-Piece Drop Pose . . . 24
8.8 MoveIt Motion Planning . . . 24
8.9 Robotiq Gripper Control . . . 25

9 ROS2 Node Architecture **26**
9.1 Node and Topic Table . . . 26
9.2 Important Topics . . . 28

10 Software Packages and Dependencies **29**
10.1 Main Packages . . . 29
10.2 Project-Specific Files . . . 30

11 Robot Configuration **31**
11.1 Important Parameters . . . 31
11.2 Example YAML Configuration . . . 31

12 Running the System **33**
12.1 Correct Running Order . . . 33
12.2 Launch Robot Driver . . . 33
12.3 Launch MoveIt . . . 34
12.4 Run Robotiq Gripper Adapter . . . 34
12.5 Run Robot Commander . . . 34
12.6 Confirm Startup Calibration . . . 34
12.7 Run Vision and Chess Nodes . . . 34

13 Testing and Results **35**
13.1 Mapper and State-Change Test . . . 35
13.2 Robot Empty Destination Test . . . 35
13.3 Robot Capture Test . . . 35
13.4 System Result . . . 36

14 Integration Challenges and Solutions **37**
14.1 Unit Mismatch Between Vision and Mapper . . . 37
14.2 Out-of-Bounds Coordinate Handling . . . 37
14.3 Vision and Robot Synchronization . . . 38
14.4 FEN Timing and Capture Handling . . . 38

15 Team Contribution **39**

16 Limitations and Future Work **40**
16.1 Current Limitations . . . 40
16.2 Future Work . . . 40

17 Conclusion **41**

A Important ROS2 Commands **42**

List of Figures

2.1 Combined system view showing YOLO detection, chess interface, and robot setup. 4

3.1 Full hardware setup showing the camera mounted above the chessboard. 6
3.2 Chessboard setup with four ArUco markers used for camera-to-board calibration. 7
3.3 Chessboard before the detected move. 8
3.4 Chessboard after the detected move. 8

5.1 Roboflow dataset version showing the final dataset split and augmentation settings. 13
5.2 YOLO detection output showing detected chess pieces and their x,y positions. 13

6.1 Terminal output showing the mapper and state-change detection result. . 16

7.1 Chess GUI showing the updated board state. 19

8.1 Robot setup in RViz showing the robot model and chessboard workspace. 22

List of Tables

9.1 ROS2 nodes, topics, and responsibilities. 27
9.2 Important ROS2 topics used in the project. 28

10.1 Main software packages and dependencies. 29

Chapter 1

Introduction

1.1 Project Motivation

The objective of this project is to develop an autonomous robotic system capable of playing chess against a human player. The project combines computer vision, robotics, coordinate transformation, chess logic, a chess engine, and motion planning.

Chess is a suitable application for this system because it requires perception, board-state understanding, legal move validation, decision-making, and precise robotic manipulation. Therefore, the project demonstrates a complete perception-to-action robotics pipeline.

1.2 Project Objective

The main objectives of the system are:

- Calibrate the camera using four ArUco markers placed on the chessboard.
- Detect chess pieces using YOLO.
- Estimate each detected piece position relative to the chessboard frame.
- Convert detected x,y positions into chessboard coordinates.
- Detect the human move by comparing previous and current board states.
- Update the chess interface and publish the FEN board representation.
- Use Stockfish to calculate the best robot move.
- Send the best move to the robot commander.
- Convert the robot move into Cartesian poses in the robot chessboard frame.
- Execute the move using the UR5e robot and Robotiq gripper.

1.3 System Scope

The system focuses on the complete integration pipeline. The main subsystems are:

1. Camera calibration using ArUco markers.
2. YOLO-based chess piece detection.
3. Mapper and state-change detection.
4. Chess interface and FEN publishing.
5. Stockfish game engine integration.
6. UR5e robot commander.
7. MoveIt-based robot motion planning.
8. Robotiq gripper control.

Chapter 2

System Overview

2.1 System Workflow

The project works according to the following order:

```
1. Camera calibration using four ArUco markers
2. YOLO detects each chess piece and its x,y position
3. Mapper converts x,y positions to chessboard squares
4. State-change node detects the human move
5. Interface updates the GUI and internal chess board
6. Interface publishes the FEN board representation
7. Stockfish receives the FEN and calculates the best move
8. Best move is sent to the interface and to the robot
9. Robot commander receives the best move on /chess_move
10. Robot converts the source and target squares into Cartesian poses
11. Robot checks FEN for capture handling
12. Robot executes the move using MoveIt and the Robotiq gripper
13. Robot publishes /robot_status for synchronization
```

2.2 High-Level Pipeline

```
Camera Calibration
-> YOLO Detection
-> Mapper
-> State Change Detection
-> Chess Interface
-> FEN Publisher
-> Stockfish Game Engine
-> Robot Commander
-> MoveIt Motion Planning
-> Robotiq Gripper Execution
```

2.3 System Demonstration

Figure 2.1 shows a combined system view including YOLO detection, the chess interface, and the robot setup.

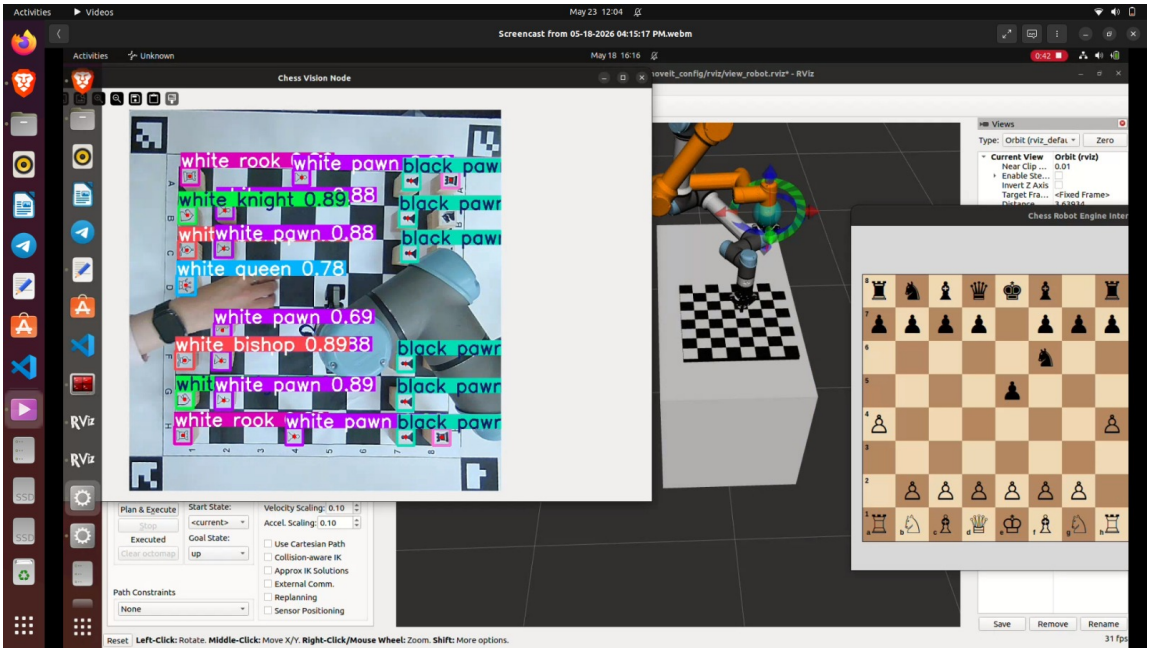


Figure 2.1: Combined system view showing YOLO detection, chess interface, and robot setup.

Chapter 3

Hardware Setup

3.1 Full Hardware Setup

The physical setup consists of a camera mounted above the chessboard. The camera captures a top-view image of the board and pieces. The chessboard contains four ArUco markers used for camera-to-board calibration.



Figure 3.1: Full hardware setup showing the camera mounted above the chessboard.

3.2 Camera Setup

The camera is mounted above the chessboard to obtain a clear top-view image of the board and pieces. The exact camera frame orientation is not critical for the chess logic because the detected positions are transformed relative to the chessboard frame.

3.3 Chessboard Setup

The chessboard has dimensions:

```
Board size: 50 cm x 50 cm
Square size: 6.25 cm x 6.25 cm
```

The chessboard frame is defined as:

```
Origin: bottom-left corner of A1
+x direction: a-file to h-file
+y direction: rank 1 to rank 8
```

3.4 Chessboard with ArUco Markers

Four ArUco markers are placed on the chessboard/workpiece. These markers are used during camera calibration to estimate the relationship between the camera and the chessboard frame.

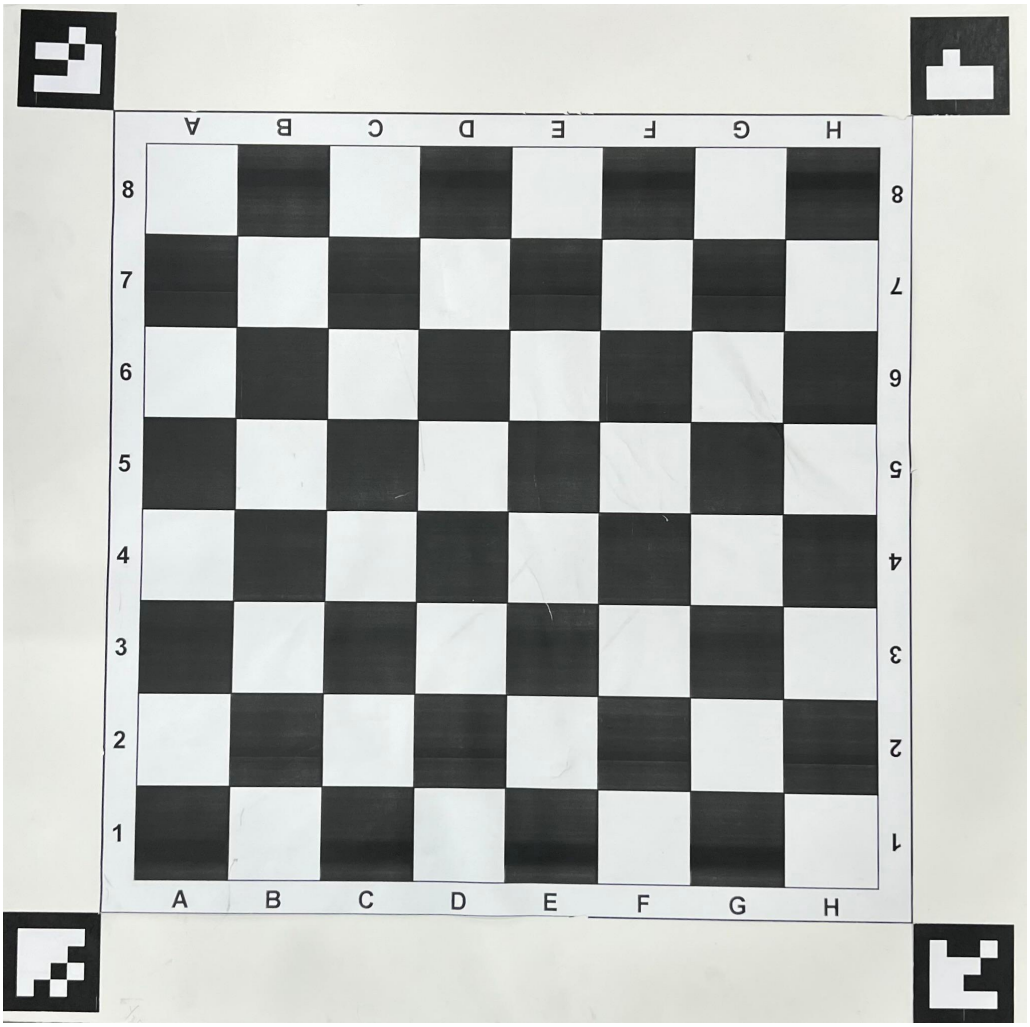


Figure 3.2: Chessboard setup with four ArUco markers used for camera-to-board calibration.

3.5 Before and After Move

The following figures show the board before and after a move. These images are used to explain the state-change detection concept.

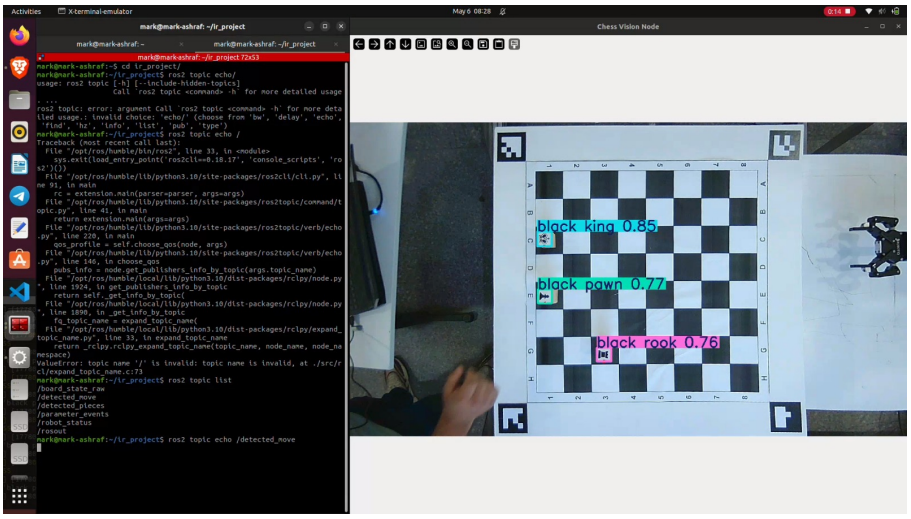


Figure 3.3: Chessboard before the detected move.

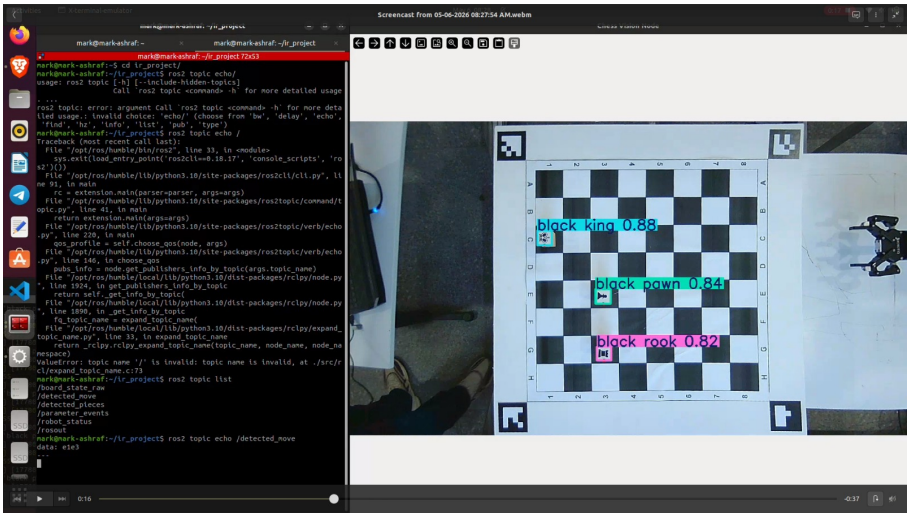


Figure 3.4: Chessboard after the detected move.

3.6 Chess Pieces

The pieces used in the project are wooden chess pieces with approximate dimensions:

```
3 cm x 3 cm x 6 cm
```

These dimensions are important for robot grasping, collision avoidance, and defining safe pick-and-place heights.

Chapter 4

Coordinate Frames and Calibration

4.1 Coordinate Frames

The system uses three main coordinate frames:

- **Camera Frame:** Used for image acquisition and initial visual detection.
- **Chessboard Frame / Workpiece Frame:** Used for mapping detected piece positions to chess squares.
- **Robot Base Frame:** Used by the robot and MoveIt for motion planning.

4.2 Chessboard Frame

The chessboard frame is fixed to the board. Its origin is located at the bottom-left corner of A1.

```
Origin: A1 bottom-left corner
+x: a -> h
+y: 1 -> 8
```

4.3 Camera Calibration Using Four ArUco Markers

The project starts with camera calibration using four ArUco markers placed on the chessboard. The purpose of this step is to calculate the relationship between the camera view and the chessboard frame. This makes it possible to convert YOLO detections from image-based positions into real x,y positions relative to the board.

After calibration, each detected piece can be represented as:

```
piece name + x position + y position
```

For example:

```
white_pawn, x = 0.281, y = 0.090
```

4.4 Manual Robot-to-Board Calibration

In this project, the robot-to-board relationship is calibrated manually. The robot first moves to the A1 square during startup calibration. At this point, the board alignment is checked and adjusted manually in RViz. After alignment is finished, a confirmation message is published on:

```
/calibration_done
```

The robot then returns to its home position and starts normal chess operation.

4.5 Startup Calibration Confirmation

The calibration is confirmed using:

```
ros2 topic pub --once /calibration_done std_msgs/msg/String "{data: 'done'}"
```

Chapter 5

Vision System and Mapping Architecture

5.1 Vision Pipeline

The vision system uses a top-down camera setup to provide a continuous view of the chessboard. The camera is used to detect chess pieces and estimate their positions relative to the chessboard frame.

The vision pipeline consists of three main stages:

1. Camera-to-board calibration using four ArUco markers.
2. YOLO-based chess piece detection.
3. Coordinate conversion from detected image positions to real x,y positions on the chessboard.

The four ArUco markers act as fixed reference points around the chessboard. They are used to estimate the relationship between the camera view and the chessboard frame. This allows the system to convert visual detections into real-world coordinates that can later be used by the mapper.

5.2 YOLO Detection Node

The YOLO node detects chess pieces from the camera image. Each detected piece is represented by its class name and its x,y position in the chessboard frame.

The YOLO detection output is published on:

```
/detected_pieces
```

The message type is:

```
std_msgs/String
```

The string contains a JSON array, for example:

```
[
  {"piece": "white_pawn", "x": 0.03125, "y": 0.09375},
  {"piece": "black_king", "x": 0.28125, "y": 0.40625}
]
```

The x and y values are expressed in meters relative to the chessboard frame.

5.3 YOLO Dataset and Training Setup

The chess piece detection model was trained using a custom dataset prepared on Roboflow. The dataset contains images of the wooden chess pieces placed on the chessboard under different positions and visual conditions.

The detection model was trained using YOLOv11 on the Roboflow dataset. A confidence threshold of 0.60 was used to reduce false detections caused by shadows, background noise, or unclear piece boundaries.

The final dataset version contained:

- Total images: 1158 images.
- Training set: 1044 images.
- Validation set: 76 images.
- Test set: 38 images.

The images were resized to:

```
640 x 640 pixels
```

Several augmentation techniques were applied to improve the robustness of the model, including:

- Horizontal and vertical flipping.
- 90-degree rotations.
- Small random rotations.
- Exposure adjustment.
- Slight blur.

These augmentations help the model generalize better to lighting variations, small camera changes, and different piece positions.

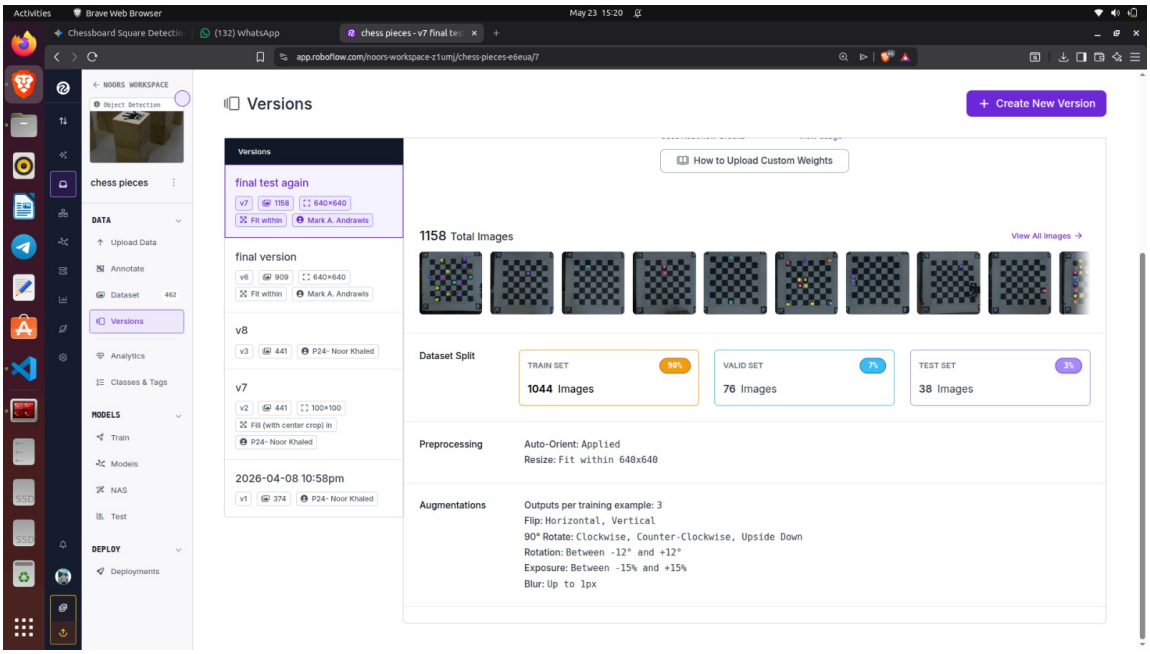


Figure 5.1: Roboflow dataset version showing the final dataset split and augmentation settings.

5.4 YOLO Detection Result

Figure 5.2 shows the YOLO detection output with calculated x,y positions.

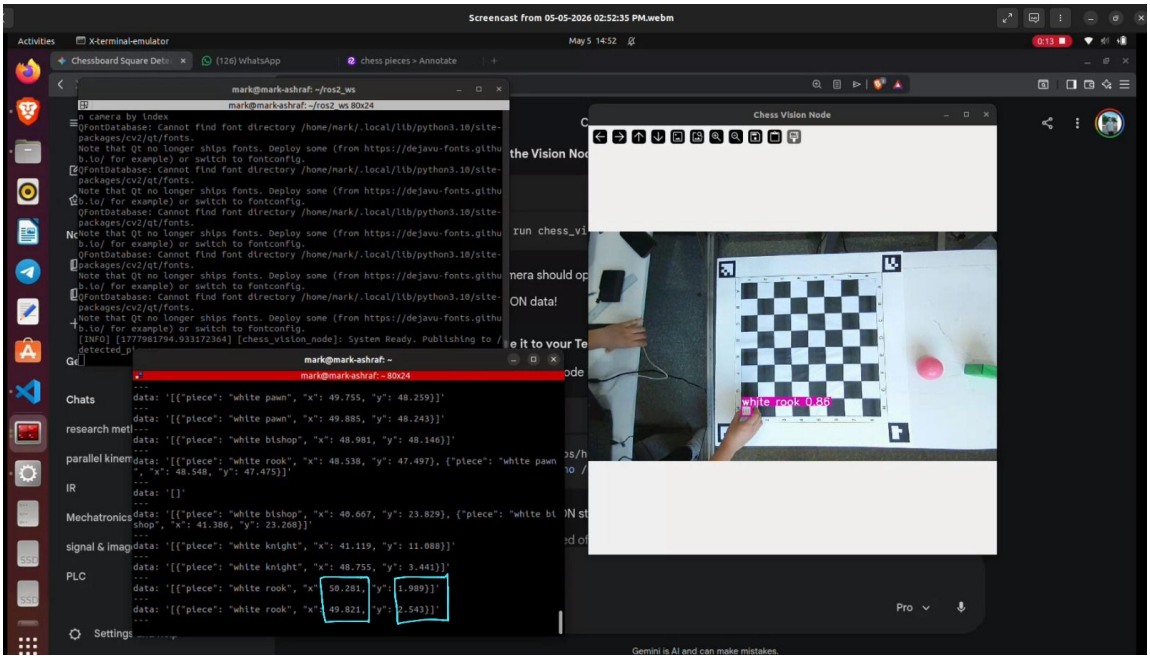


Figure 5.2: YOLO detection output showing detected chess pieces and their x,y positions.

5.5 Coordinate Mapping and Boundary Clamping

To convert real-world x,y coordinates into chess notation, the mapper uses a grid-mapping algorithm. The square size is:

$$S = 0.0625 \text{ m} \tag{5.1}$$

The chess square is determined by calculating the column and row indices:

$$col = \left\lfloor \frac{x}{S} \right\rfloor \tag{5.2}$$

$$row = \left\lfloor \frac{y}{S} \right\rfloor \tag{5.3}$$

Since the physical board size is 0.50 m by 0.50 m, values outside this range are invalid. During integration, it was observed that the vision node could sometimes send values slightly larger than the board size due to calibration error or detection noise. Therefore, boundary clamping was added to ensure the coordinates remain inside the valid board range.

```
BOARD_SIZE = 0.50
EPSILON = 1e-6

x = min(max(x, 0.0), BOARD_SIZE - EPSILON)
y = min(max(y, 0.0), BOARD_SIZE - EPSILON)
```

The small epsilon is important because sending exactly 0.50 m would result in:

```
floor(0.50 / 0.0625) = 8
```

which is outside the valid chessboard index range from 0 to 7.

Chapter 6

Mapper and State-Change Detection

6.1 Mapper Concept

The mapper converts real x,y positions in the chessboard frame into chessboard squares.

Each square has a size of:

$$S = 0.0625 \text{ m} \tag{6.1}$$

The column and row are calculated as:

$$col = \left\lfloor \frac{x}{S} \right\rfloor \tag{6.2}$$

$$row = \left\lfloor \frac{y}{S} \right\rfloor \tag{6.3}$$

The column is converted to a chess file:

```
0 -> a
1 -> b
2 -> c
...
7 -> h
```

The row is converted to a rank:

```
0 -> 1
1 -> 2
...
7 -> 8
```

6.2 Mapper Example

If YOLO detects:

```
piece = white_pawn
x = 0.281 m
```

```
y = 0.090 m
```

Then:

```
col = floor(0.281 / 0.0625) = 4 -> e
row = floor(0.090 / 0.0625) = 1 -> rank 2
```

Therefore, the mapper identifies:

```
white_pawn in e2
```

6.3 State-Change Detection

The state-change node compares the previous board state with the current board state.

Example before the move:

```
{
  "e2": "white_pawn"
}
```

Example after the move:

```
{
  "e4": "white_pawn"
}
```

The system detects:

```
e2: white_pawn -> empty
e4: empty -> white_pawn
```

Therefore, the detected move is:

```
e2e4
```

6.4 Mapper and State-Change Terminal Result

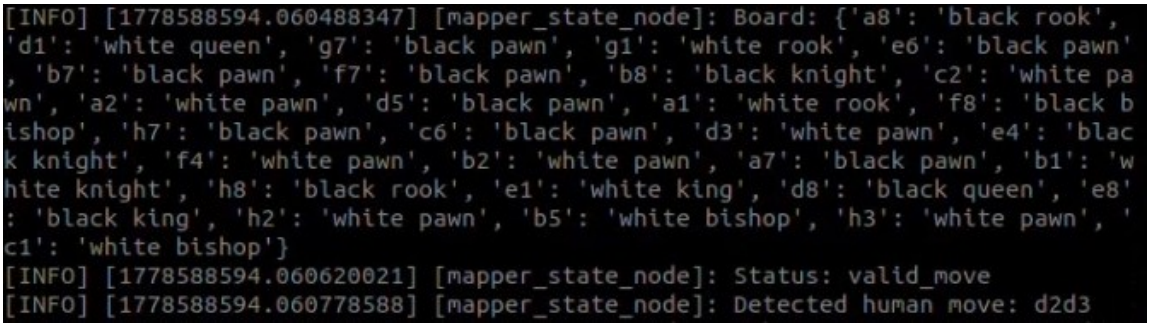


Figure 6.1: Terminal output showing the mapper and state-change detection result.

6.5 Robot Status Synchronization

The mapper uses the robot status topic to avoid detecting the robot move as a human move.

The robot publishes:

```
/robot_status
```

Possible values:

```
moving
idle
```

When the robot is moving, the vision and mapper system should avoid updating the board state. When the robot returns to idle, the next stable frame can be used to synchronize the board.

Chapter 7

Chess Interface and Game Engine

7.1 Chess Interface

The chess interface receives the detected human move and applies it to the internal chess board. It updates the GUI, checks the state of the game, and publishes the FEN representation of the board.

The detected move is received from:

```
/detected_move
```

The interface publishes FEN on:

```
/interface/board_fen
```

7.2 Chess GUI Result



Figure 7.1: Chess GUI showing the updated board state.

7.3 FEN Representation

FEN is a compact text representation of a chess position. It contains:

- Piece placement.
- Side to move.
- Castling rights.
- En passant information.
- Halfmove clock.
- Fullmove number.

Example FEN after `e2e4`:

```
rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq - 0 1
```

7.4 Stockfish Game Engine

The game engine receives the FEN from the interface and calculates the best move.

The engine output is sent to:

```
/chess_move
```

This topic is subscribed by the robot commander.

Example best move:

```
a2a4
```

The selected engine move is also displayed on the interface for visualization.

Chapter 8

Robot System

8.1 Robot Commander Node

The main robot node is responsible for controlling the UR5e robot using MoveIt. It receives the chess move from the game engine, converts the source and target chess squares into robot poses, checks if the target square is occupied, and executes the move.

8.1.1 Main Node

```
ee_pose_commander
```

8.1.2 Subscribed Topics

```
/chess_move
/interface/board_fen
/calibration_done
```

8.1.3 Published Topic

```
/robot_status
```

8.2 Robot Setup in RViz

The robot is configured in RViz together with the chessboard model. During startup, the robot moves to A1 for manual alignment. After calibration is confirmed, the robot returns home and starts receiving chess moves.

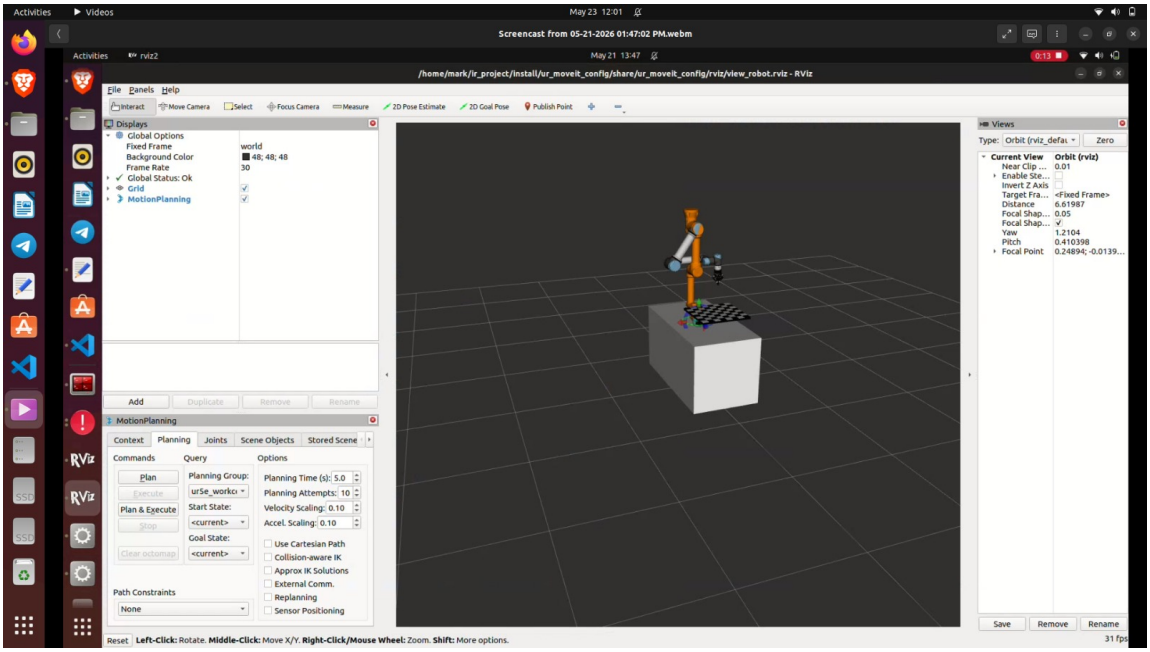


Figure 8.1: Robot setup in RViz showing the robot model and chessboard workspace.

8.3 Robot Startup Calibration

The robot startup sequence is:

1. Move to the home target.
2. Move above A1.
3. Move down to the actual A1 calibration pose.
4. Wait for manual board alignment.
5. Wait until `/calibration_done` receives `done`.
6. Move back above A1.
7. Return home.
8. Start normal chess operation.

The calibration confirmation command is:

```
ros2 topic pub --once /calibration_done std_msgs/msg/String "{data: 'done'}"
```

8.4 Square-to-Pose Mapping

The robot receives moves in chess notation, such as:

```
a2a4
```

The first two characters represent the source square, and the next two characters represent the target square.

The robot converts the square to row and column:

```
file a -> column 0
rank 2 -> row 1
```

Then it calculates the center of the square:

$$x = \frac{square_size}{2} + col \times square_size \tag{8.1}$$

$$y = \frac{square_size}{2} + row \times square_size \tag{8.2}$$

where:

$$square_size = \frac{board_size}{8} \tag{8.3}$$

For a 0.50 m board:

$$square_size = 0.0625 \text{ m} \tag{8.4}$$

8.5 Normal Robot Move Execution

The normal pick-and-place sequence is:

1. Move above the source square.
2. Move down to the source square.
3. Close the gripper.
4. Lift the piece.
5. Move above the target square.
6. Move down to the target square.
7. Open the gripper.
8. Move back above the target square.
9. Return home if enabled.

8.6 Capture Handling

Before executing a robot move, the robot checks the FEN from:

```
/interface/board_fen
```

The purpose is to know whether the target square is occupied.
If the destination square is empty, the robot performs the normal move directly.
If the destination square is occupied, the robot performs the following sequence:

1. Move to the occupied target square.
2. Pick the captured piece.
3. Move to the captured-piece drop pose.
4. Release the captured piece outside the board.
5. Return home.
6. Execute the normal chess move.

8.7 Captured-Piece Drop Pose

The captured-piece drop pose is expressed in:

```
base_link
```

Example drop pose parameters:

```
capture_drop_x = -0.162
capture_drop_y = -0.055
capture_drop_z = 1.207
```

8.8 MoveIt Motion Planning

The robot uses MoveIt to plan and execute movements. The main planning group is:

```
ur5e_workcell_manipulator
```

The chess square poses are expressed in:

```
chess_board_frame
```

The end-effector link is:

```
tool0
```

8.9 Robotiq Gripper Control

The gripper is controlled through the Robotiq 2F adapter node. Before running the robot commander with the real gripper, the gripper adapter is started using:

```
ros2 run robotiq_2f_urcap_adapter robotiq_2f_adapter_node.py --ros-args -p robot_ip:=192.168.1.102
```

The gripper action name used by the robot commander is:

```
/robotiq_2f_urcap_adapter/gripper_command
```

Typical gripper positions:

```
gripper_open_position  = 0.050
gripper_close_position = 0.037
```

Chapter 9

ROS2 Node Architecture

9.1 Node and Topic Table

Table 9.1: ROS2 nodes, topics, and responsibilities.

Node	**Subscribes To / Inputs**	**Publishes To / Outputs**	**Description**
Camera Calibration	Camera frames	Calibration transform	Uses four ArUco markers to calibrate the camera-to-board relation.
YOLO Detection Node	Camera image	`/detected_pieces`	Detects chess pieces and estimates their x,y positions.
Mapper Node	`/detected_pieces`	`/board_state_raw`	Converts x,y positions to chessboard squares.
State-Change Node	`/board_state_raw`, `/robot_status`	`/detected_move`	Detects the human move by comparing board states.
Chess Interface	`/detected_move`, `/chess_move`	`/interface/board_fen`	Updates the GUI and publishes the FEN board state.
Stockfish Engine	`/interface/board_fen`	`/chess_move`	Calculates the best move.
Robot Commander	`/chess_move`, `/interface/board_fen`, `/calibration_done`	`/robot_status`	Executes robot moves and handles captures.
Robotiq Adapter	Gripper action request	Gripper action feedback/result	Controls the Robotiq 2F gripper.

9.2 Important Topics

Table 9.2: Important ROS2 topics used in the project.

Topic	**Description**
`/detected_pieces`	YOLO output containing piece labels and x,y positions.
`/board_state_raw`	Current board state after mapping.
`/detected_move`	Human move detected from state change.
`/interface/board_fen`	FEN board state published by the chess interface.
`/chess_move`	Best robot move sent from the engine to the robot commander.
`/calibration_done`	Confirmation topic used after manual A1 calibration.
`/robot_status`	Robot status used to synchronize vision and robot motion.

Chapter 10

Software Packages and Dependencies

10.1 Main Packages

Table 10.1: Main software packages and dependencies.

Package	**Purpose**	**Installation / Source**
ROS2	Node communication and system integration.	Official ROS2 installation guide.
OpenCV	Image processing and ArUco detection.	`pip install opencv-python opencv-contrib-python`
NumPy	Matrix and transformation operations.	`pip install numpy`
Ultralytics YOLO	Chess piece detection.	`pip install ultralytics`
python-chess	Chess rules, legal moves, and FEN generation.	`pip install chess`
Stockfish	Chess engine for best move calculation.	`sudo apt install stockfish`
MoveIt2	Robot motion planning.	Official MoveIt2 installation guide.
RViz	Robot and chessboard visualization.	Installed with ROS2 / MoveIt setup.
Robotiq 2F URCap Adapter	Control Robotiq gripper.	Project package / ROS2 package.

10.2 Project-Specific Files

- `camera_matrix.npy`: Camera intrinsic matrix.
- `dist_coeffs.npy`: Camera distortion coefficients.
- `best.pt`: YOLO trained model file.
- `robot_config.yaml`: Robot and system parameters.
- `ik_commander.launch.py`: Robot commander launch file.

Chapter 11

Robot Configuration

11.1 Important Parameters

The robot commander uses several parameters to control the board size, motion height, planning group, frames, speed, gripper, and capture behavior.

```
board_size: 0.50
pick_z: 0.2
lift_z: 0.2
planning_group: ur5e_workcell_manipulator
pose_reference_frame: chess_board_frame
end_effector_link: tool0
home_named_target: up

startup_calibration_enabled: true
calibration_done_topic: /calibration_done
board_fen_topic: /interface/board_fen
chess_move_topic: /chess_move
robot_status_topic: /robot_status

capture_drop_frame: base_link
capture_drop_x: -0.162
capture_drop_y: -0.055
capture_drop_z: 1.207

use_gripper: true
gripper_action_name: /robotiq_2f_urcap_adapter/gripper_command
gripper_open_position: 0.050
gripper_close_position: 0.037
```

11.2 Example YAML Configuration

```
robot:
  name: "UR5e"
  planning_group: "ur5e_workcell_manipulator"
  base_frame: "base_link"
  pose_reference_frame: "chess_board_frame"
  end_effector_link: "tool0"
```

```
  home_named_target: "up"

chessboard:
  board_size: 0.50
  square_size: 0.0625
  frame: "chess_board_frame"
  origin: "bottom-left corner of A1"

motion:
  pick_z: 0.2
  lift_z: 0.2
  velocity_scale: 0.40
  acceleration_scale: 0.40
  planning_time: 10.0
  num_planning_attempts: 10
  return_home_after_move: true

topics:
  detected_pieces: "/detected_pieces"
  board_state_raw: "/board_state_raw"
  detected_move: "/detected_move"
  board_fen: "/interface/board_fen"
  chess_move: "/chess_move"
  robot_status: "/robot_status"
  calibration_done: "/calibration_done"

capture:
  drop_frame: "base_link"
  drop_x: -0.162
  drop_y: -0.055
  drop_z: 1.207

gripper:
  use_gripper: true
  action_name: "/robotiq_2f_urcap_adapter/gripper_command"
  open_position: 0.050
  close_position: 0.037
  max_speed: 0.10
  max_effort: 50.0
```

Chapter 12

Running the System

12.1 Correct Running Order

The recommended running order is:

1. Launch the UR robot driver.
2. Launch MoveIt.
3. Run the Robotiq gripper adapter.
4. Run the robot commander.
5. Wait until the robot reaches the A1 calibration pose.
6. Confirm startup robot calibration using `/calibration_done`.
7. Run camera calibration or load the saved calibration.
8. Run the YOLO detection node.
9. Run the mapper and state-change node.
10. Run the chess interface.
11. Run the Stockfish game engine node.

12.2 Launch Robot Driver

```
ros2 launch ur_robot_driver ur_control.launch.py ur_type:=ur5e robot_ip:=192.168.1.102 use_fake_hardware:=
    true
```

For the real robot, `use_fake_hardware` should be changed carefully only when the setup is safe.

12.3 Launch MoveIt

```
ros2 launch ur_moveit_config ur_moveit.launch.py ur_type:=ur5e robot_ip:=192.168.1.102 use_fake_hardware:=
    true
```

12.4 Run Robotiq Gripper Adapter

```
ros2 run robotiq_2f_urcap_adapter robotiq_2f_adapter_node.py --ros-args -p robot_ip:=192.168.1.102
```

12.5 Run Robot Commander

```
ros2 launch ur5e_grip_run_cpp ik_commander.launch.py
```

12.6 Confirm Startup Calibration

After the robot reaches A1 and the manual alignment is finished, publish:

```
ros2 topic pub --once /calibration_done std_msgs/msg/String "{data: 'done'}"
```

12.7 Run Vision and Chess Nodes

Use the following placeholders and replace them with the actual commands used in the project:

```
# Run camera calibration or camera node
ros2 run chess_vision_pkg calibrate_homography

# Run YOLO detection node
ros2 run chess_vision_pkg vision_node

# Run mapper and state-change node
ros2 run chess_mapper_state mapper_state_node

# Run chess interface with Stockfish engine node
ros2 run chess_engine_bridge detected_moves_engine_node
```

Chapter 13

Testing and Results

13.1 Mapper and State-Change Test

The mapper and state-change node can be tested manually by publishing fake detections.

Initial board:

```
ros2 topic pub --once /detected_pieces std_msgs/msg/String "data: '[{\"piece\":\"white_pawn\",\"x
    \":0.03125,\"y\":0.09375}]'"
```

Move from A2 to A3:

```
ros2 topic pub --once /detected_pieces std_msgs/msg/String "data: '[{\"piece\":\"white_pawn\",\"x
    \":0.03125,\"y\":0.15625}]'"
```

Expected output:

```
/detected_move = a2a3
```

13.2 Robot Empty Destination Test

Publish a FEN where the source square is occupied and the destination is empty:

```
ros2 topic pub --once /interface/board_fen std_msgs/msg/String "{data: 'rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP
    /RNBQKBNR w KQkq - 0 1'}"
```

Then publish the robot move:

```
ros2 topic pub --once /chess_move std_msgs/msg/String "{data: 'e2e4'}"
```

Expected result:

```
The robot executes the normal pick-and-place move from e2 to e4.
```

13.3 Robot Capture Test

Publish a FEN where the destination square is occupied:

```
ros2 topic pub --once /interface/board_fen std_msgs/msg/String "{data: 'rnbqkbnr/pppppppp/8/8/4p3/8/
    PPPPPPPP/RNBQKBNR w KQkq - 0 1'}"
```

Then publish the robot move:

```
ros2 topic pub --once /chess_move std_msgs/msg/String "{data: 'e2e4'}"
```

Expected result:

```
The robot removes the piece on e4 first, drops it outside the board, returns home, and then moves the
    piece from e2 to e4.
```

13.4 System Result

The system was able to detect piece positions, map them to chess squares, identify a human move, update the interface, publish FEN, generate a Stockfish best move, and send the robot move to the UR5e commander.

Chapter 14

Integration Challenges and Solutions

14.1 Unit Mismatch Between Vision and Mapper

One of the first integration problems was a unit mismatch between the vision node and the mapper/state-change node. The vision system initially produced coordinates in centimeters, while the mapper expected the input coordinates in meters.

This caused incorrect square mapping because the mapper divides the x and y values by the square size:

$$S = 0.0625 \text{ m} \tag{14.1}$$

Therefore, if a coordinate was sent as 28 cm instead of 0.28 m, the mapper interpreted the value as 28 meters, which is far outside the chessboard.

The solution was to convert the vision output from centimeters to meters before publishing it to the mapper:

```
x_m = x_cm / 100.0
y_m = y_cm / 100.0
```

After this conversion, the mapper received coordinates in the correct unit and was able to map pieces to the correct chess squares.

14.2 Out-of-Bounds Coordinate Handling

Another issue occurred when the vision system sometimes produced x or y values slightly outside the physical board dimensions. Since the board size is:

```
0.50 m x 0.50 m
```

any coordinate greater than 0.50 m is outside the chessboard workspace.

This can happen due to detection noise, perspective error, or small calibration inaccuracies near the board edges. To prevent invalid square mapping, a boundary clamping

step was added.

```
BOARD_SIZE = 0.50
EPSILON = 1e-6

x = min(max(x, 0.0), BOARD_SIZE - EPSILON)
y = min(max(y, 0.0), BOARD_SIZE - EPSILON)
```

This ensures that all published coordinates remain inside the board limits and prevents the mapper from generating invalid squares.

14.3 Vision and Robot Synchronization

During robot motion, the camera may see the robot arm, gripper, motion blur, or a chess piece between two squares. This can cause unstable detections and incorrect state-change results.

To solve this issue, the robot commander publishes its motion state on:

```
/robot_status
```

When the robot publishes:

```
moving
```

the mapper avoids updating the board state. When the robot publishes:

```
idle
```

the vision system can safely synchronize the final board state again.

14.4 FEN Timing and Capture Handling

Another integration issue was related to capture moves. The robot needs to know whether the destination square is occupied before physically moving a piece. For this reason, the robot commander subscribes to:

```
/interface/board_fen
```

Before executing a move received on:

```
/chess_move
```

the robot checks the FEN board state. If the destination square is occupied, the robot first removes the captured piece and drops it outside the board. After that, it returns home and performs the normal chess move.

This makes the robot capable of handling both normal moves and capture moves correctly.

Chapter 15

Team Contribution

The project was divided into several main tasks, including vision detection, board mapping, chess interface development, game engine integration, robot setup, and robot motion execution. Each subsystem was tested individually and then integrated into the full ROS2 pipeline.

- **Vision and calibration:** Camera setup, ArUco marker calibration, YOLO dataset preparation, model training, and piece detection.
- **Mapping and state change:** Conversion of detected piece positions into chessboard squares and detection of human moves.
- **Interface and game engine:** GUI update, FEN publishing, and Stockfish integration.
- **Robot system:** UR5e setup, RViz and MoveIt configuration, gripper integration, startup calibration, capture handling, and robot move execution.

Chapter 16

Limitations and Future Work

16.1 Current Limitations

- YOLO may confuse piece types or colors.
- Pieces near square borders may be ignored due to margin filtering.
- The system still requires tuning for margin, minimum movement distance, and processing interval.
- Special chess cases such as castling, promotion, and en passant may require extra handling.
- Manual robot-to-board calibration must be repeated if the board or robot setup changes.

16.2 Future Work

- Improve the YOLO dataset and detection accuracy.
- Add confidence filtering for YOLO detections.
- Add majority voting over multiple frames.
- Improve automatic robot-to-workpiece calibration.
- Publish transforms using ROS2 TF2.
- Add collision objects for pieces in the motion planner.
- Add full support for castling, promotion, and en passant.
- Improve the GUI to display detected moves, engine moves, robot status, and capture status.

Chapter 17

Conclusion

This project demonstrates a complete autonomous chess robot pipeline that integrates perception, chess logic, game-engine decision-making, and robot execution. The system starts with camera calibration using four ArUco markers, then uses YOLO to detect chess pieces and their positions. The mapper converts these positions to chessboard coordinates, and the state-change logic detects the human move.

The interface updates the board, publishes the FEN, and sends the current game state to Stockfish. The game engine returns the best move, which is visualized in the interface and sent to the robot commander. The robot converts the chess move into Cartesian poses in the chessboard frame and executes the motion using MoveIt and the Robotiq gripper.

The modular ROS2 architecture allows each part of the system to be developed and tested separately. The robot commander also includes important practical features such as startup calibration at A1, FEN-based capture handling, and robot-status synchronization with the vision system. During integration, practical issues such as unit mismatch and out-of-bound vision coordinates were solved to make the full pipeline more reliable.

Appendix A

Important ROS2 Commands

```
# Run Robotiq gripper adapter
ros2 run robotiq_2f_urcap_adapter robotiq_2f_adapter_node.py --ros-args -p robot_ip:=192.168.1.102

# Confirm startup calibration
ros2 topic pub --once /calibration_done std_msgs/msg/String "{data: 'done'}"

# Test robot move
ros2 topic pub --once /chess_move std_msgs/msg/String "{data: 'e2e4'}"

# Echo robot status
ros2 topic echo /robot_status

# Echo board FEN
ros2 topic echo /interface/board_fen
```